

Lab04 - Trains

Général

- Auteurs: Fabien Léger & Aymeric Bonny
- Cours: PCO, HEIG-VD
- Date: 25.11.2025

Problème imposé

Il nous est demandé d'implémenter deux locomotives qui roulent sur des rails. Les rails s'entrecroisent à différents endroits d'un plan 2D. Le tout forme plusieurs boucles. Il faut choisir des tracés tels que les deux locomotives se croisent sur un rail formant une section commune.

Le but du laboratoire est donc de mettre en place ces tracés et d'implémenter la section partagée pour qu'à n'importe quel moment donné, seul une locomotive se trouve dans la section partagée physiquement.

Implémentation

L'implémentation du programme s'est passé en différentes étapes

- Choix de la maquette, des aiguillages et des chemins de base
- Mise en place de la SharedSection
- Ajout de paramètres dans cmain
- Mise en place de LocomotiveBehaviour::run
- Ajout de emergency_stop

Choix de la maquette, des aiguillages et des chemins de base

Nous avons choisi la maquette B, car nous trouvions le circuit plus intéressant dans le cas où nous aurions plus de temps. Nous voulions également apprendre comment fonctionnent la création des aiguillages, contacts, etc.

Les aiguillages ont été mis en place pour faciliter nos chemins. La section partagée n'est pas très importante du point de vue de quel aiguillage est mis comment, car ceux-ci sont checkés avant d'entrer.

Le chemin suivi par les locomotives est le suivant

- LocoA (rouge) : {18, 17, 12, 11, 4, 5, 24, 23}
- LocoB (bleu) : {13 10, 11, 4, 3, 19, 14}

Mise en place de la SharedSection

Le moyen le plus compréhensible d'implémenter est d'avoir les variables suivantes:

```
PcoSemaphore mutex = PcoSemaphore(1); // protects occupied decision
making and nbWaiting
PcoSemaphore left = PcoSemaphore(0); // if section is occupied, makes
other locomotives wait (barrier)
bool occupied = false; // true if there is a locomotive in
the shared section, false otherwise
int nbWaiting = 0; // number of locomotives waiting to
access shared section
int countErrors = 0; // count the number of errors
because of wrong calls to SharedSection
Locomotive *loco = nullptr; // current loco in the section
Direction direction = Direction::D1; // direction of the loco inside the
section (by default D1)
bool stopped = false; // used to stop all locomotives
```

La première locomotive qui accède à la section change **occupied** à true et se met comme locomotive utilisatrice via la variable **loco** et **direction**.

On peut ainsi bloquer les locomotives qui rentrent dans la section partagée avec **left** en les comptant avec **nbWaiting**.

Lorsque la première locomotive leave, la seule action est que **occupied** passe à false.

Finalement, notre première locomotive quitte la section.

Dans le cas où **nbWaiting** est plus grand que 0, on reboucle la section partagée avant de release **left** pour laisser aller la prochaine locomotive.

Il est important de toujours faire attention aux variables concurrentes en les protégeant avec notre mutex.

countErrors sert à attraper des mauvaises utilisations de SharedSection.

Ajout de paramètres dans cmain

Dans le main, nous avons rajouté deux arrays, qui contiennent 4 éléments. Ces éléments sont les points de contacts importants du parcours que nous avons sélectionné pour ces deux trains. Ils contiennent le point de contact du leave et du release, dans les deux sens (le point de leave correspond au point d'accès si nous allons dans l'autre sens). Nous avons aussi rajouté comme arguments au constructeur de locomotiveBehavior cet array et la direction de départ de la locomotive.

Mise en place de LocomotiveBehaviour::run

La fonction run est cadencé par les appels à attendre_contact() et sharedSection->access(). J'ai créé deux fonctions supplémentaires. La première, nextPoint() sert à renvoyer le prochain point de contact sur lequel appeler attendre_contact, tandis que la deuxième, directionChange(), sert à créer un booléen random pour décider de changer de sens ou non.

Pour ce qui est des accès concurrents, je me suis servi de la semaphore de access pour protéger une variable qui alterne les aiguillages. C'est une variable partagée, mais comme les deux threads ne peuvent pas se trouver entre access et release en même temps, elle est protégée. J'ai aussi créé une sémaphore supplémentaire pour protéger un array contenant les directions des deux trains, de manière à les comparer et ainsi décider du comportement à adopter en arrivant à leave.

Ajout de emergency_stop

L'important ici était de d'abord relâcher tous les threads bloqués dans SharedSection via stopAll avant d'arrêter les locomotives. Dans le cas contraire, on pourrait avoir des locomotives qui se font arrêter par notre emergency_stop mais qui se font relancer pas un release d'une autre locomotive.

L'utilité de stopAll est de changer le bool **stopped** à true avant de release les threads. La première chose que ces threads feront est de checker cette variable et de quitter access si c'est le cas. Cela pourrait amener à des choses inattendues mais nous n'allons pas rappeler access directement et la variable stopped est également checkée en début de access au cas où.

Tests

Nous avons passé tous les tests déjà mis avec le labo.

Quelques tests ont été rajoutés afin de tester différents cas qui n'était pas prévu.

Voici un tableau récapitulatif:

Suite de tests	Nom du test	Description
SharedSectionStudent	ConsecutiveLeave_IsError	Deux appels consécutifs à leave() doivent produire une erreur.
SharedSectionStudent	ConsecutiveRelease_IsError	Deux appels consécutifs à release() doivent produire une erreur.
SharedSectionStudent	LeaveWithoutAccess_IsError	Appeler leave() sans avoir fait access() auparavant doit produire une erreur.

Suite de tests	Nom du test	Description
SharedSectionStudent	ReleaseWithoutAccess_IsError	Appeler <code>release()</code> sans avoir fait <code>access()</code> doit produire une erreur.
SharedSectionStudent	ReleaseWithoutLeave_IsError	Appeler <code>release()</code> sans avoir fait <code>leave()</code> doit produire une erreur.
SharedSectionStudent	MultipleErrors_CountCorrectly	Plusieurs erreurs successives doivent être comptabilisées correctement.
SharedSectionStudent	GoThroughSharedSectionMultipleTimes_IsNoErrors	Une locomotive traverse plusieurs fois la section sans générer d'erreurs.
TwoTrainsInteractions	TwoOppositeDirection_SerializesCorrectly	Deux locomotives de sens opposé accèdent correctement à la section.
TwoTrainsInteractions	SecondLocoLeaveOrReleaseWithoutAccess_IsError	Une deuxième locomotive qui appelle <code>leave()/release()</code> sans <code>access()</code> doit déclencher des erreurs.
TwoTrainsInteractions	SecondLocoReleaseBeforeFirstLocoRelease_IsError	Une deuxième locomotive qui appelle <code>release()</code> alors qu'elle n'a rien fait déclenche une erreur.
ThreeTrainsInteractions	ThreeTrains_SerializesCorrectly	Trois locomotives accèdent à la section normalement. (pour le fun)

Tous ces tests ont également passé. Nous nous sommes concentré sur SharedSection, car cela semblait le plus important. Il paraît difficile de toute façon de faire des tests avec le `run()` des locos.

Problèmes survenus

1. La locomotive bleu se téléporte lors du changement de direction probablement à cause de sa vitesse. Nous n'avons donc pas corrigé cette fonctionnalité, ne sachant de toute façon pas comment le faire.
2. La locomotive rouge ne passe pas sur la bonne section partagée bien que les aiguillages sont arrangés de la bonne des façons sur la GUI. Nous pensons donc que c'est également un problème d'affichage.
3. Pour des raisons qui nous sont inconnues, lorsque les locomotives ne vont pas dans le même sens, il arrive que la locomotive sortant release au second contact plutôt qu'au premier. Cela agit comme si les locomotives allaient dans le même sens alors que ce n'est pas le cas.